

SYSTEMENTOR EDUCATION

Systemmentor AB

Blåklockans väg 8

yahya.hussein@systemmentor.se

132 45 SaltsjöBoo

INLÄMNINGSUPPGIFT GRUPP (G-VG)

+

INLÄMNINGSUPPGIFT Individuellt (G)

BÅDA LÄMNAS IN PÅ SAMMA INLÄMNING

VARJE STUDENT LÄMNAR IN REPOLÄNK OCH SIN EGEN RAPPORT.

Individuella inlämningen hittar du längre ner i dokumentet.

Projekt: Pensionatet delas upp – från monolit till microservices

NI JOBBAR I SAMMA GRUPPER SOM I BACKEND 1

Länk till grupperna har ni här:

<https://docs.google.com/spreadsheets/d/117HQA9bMjf-2MzOYj2NWKf4e7Xj0T0GOln1zUeEuFE/edit?usp=sharing>

MEDDELA MIG SENAST NU PÅ FREDAG(21 augusti) OM NI INTE KAN FORTSÄTTA I ER GRUPP, SÅ LÖSER VI DET

Vad handlar uppgiften om?

I Backend 1 byggde ni ett bokningssystem för ett pensionat. Allt låg i EN enda applikation. En sådan applikation kallas för en monolit.

Pensionatet har växt och systemet ska byggas ut. Yahya har bestämt att systemet ska delas upp i mindre tjänster (microservices) som pratar med varandra.

I den här uppgiften ska ni **INTE** bygga något nytt bredvid ert pensionat. Ni ska **DELA UPP** det.

Så här ser ert system ut efter Backend 2:

- Tjänst 1: Bokningstjänsten: ert pensionat från Backend 1. Ansvarar för rum och bokningar. Frontend ligger kvar här. (Om ni kört thymeleaf)

- Tjänst 2: Kundtjänsten: en helt ny applikation. Ansvarar för ALL kundhantering.
- Tjänst 3 (endast VG): en valfri tredje tjänst (**se VG kraven**)

Tjänsterna pratar med varandra via REST. Varje tjänst har en egen databas.

Krav för Godkänt (G)

För att få betyget Godkänt (G) ska projektet uppfylla samtliga krav nedan.

1. Utgångspunkt (Förvaltning av kod = 80% av tiden som utvecklare!)

- Ni ska bygga vidare på ert pensionat från Backend 1
- Ni får rätta buggar och städa i den gamla koden
- Ni ska **INTE** börja om från noll

2. Bokningstjänsten

- Ert pensionat från Backend 1 blir nu bokningstjänsten
- Bokningstjänsten ansvarar för rum och bokningar
 - Er frontend från Backend 1 ligger kvar i bokningstjänsten (Om ni har använt thymeleaf)
- Bokningstjänsten ska **INTE** längre ha någon kundtabell i sin databas all kunddata flyttar till kundtjänsten(Tjänst/Service 2)
- En bokning sparar bara kundens id (ett nummer/email/telefonnummer osv) inte hela kunden
- Alla regler från Backend 1 gäller fortfarande, till exempel: inga dubbel bokningar på samma rum och datum

3. Kundtjänsten (den nya tjänsten/servicen)

- Kundtjänsten ska vara ett helt nytt Spring Bootprojekt (en egen applikation)
- Kundtjänsten **ska ha en EGEN databas(OBS SQLITE/H2 EJ TILLÅTET!!)**
- Kundtjänsten ansvarar för ALL kundhantering: registrera, ändra och ta bort kunder
- Kundtjänsten ska ha ett REST API som svarar med JSON, minst:
 - hämta en kund
 - hämta alla kunder (ENBART OM NI HAR EN ADMIN SIDA, ANNARS BORTSE FRÅN DENNA PUNKT!)
 - registrera en kund
 - ändra en kunds uppgifter
 - ta bort en kund

React

Email

?

DI

Isac

?

- Regeln från Backend 1 gäller fortfarande: en kund får bara tas bort om kunden inte har några aktiva bokningar
- Men nu ligger bokningarna i en annan tjänst! Kundtjänsten måste alltså FRÅGA bokningstjänsten om kunden har aktiva bokningar innan kunden tas bort

4. Kommunikation mellan tjänsterna

- Tjänsterna ska prata med varandra via REST anrop
- Tjänsterna får **ALDRIG** läsa eller skriva direkt i varandras databaser
- När en bokning skapas ska bokningstjänsten fråga kundtjänsten att kunden verkligen finns
- När en kund ska tas bort ska kundtjänsten fråga bokningstjänsten om kunden har aktiva bokningar
- API:erna ska svara med rätt statuskoder, till exempel:
 - 200 OK – det gick bra
 - 201 Created – något skapades
 - 400 Bad Request – felaktig inmatning
 - 404 Not Found – finns inte
 - 409 Conflict – till exempel dubbelbokning, eller kund med aktiva bokningar
- Om den andra tjänsten är nere ska er tjänst INTE krascha, visa ett tydligt felmeddelande istället. (tex, Vi kunde inte hantera din bokning just nu, försök igen senare)

5. Docker

- Bokningstjänsten ska ha en egen Dockerfile
- Kundtjänsten ska ha en egen Dockerfile
- Det ska finnas en dockercompose.yml som startar HELA systemet: båda tjänsterna och databaserna
- Hela systemet ska starta med ett enda kommando: docker compose up

6. Versionshantering

- Projektet ska versionshanteras i GitHub från början till slut
- Koden ska pushas löpande under arbetets gång och inte bara läggas upp i slutet
- Alla gruppmedlemmar ska bidra i repositoret

7. Deployment

- Projektet ska vara deployad på Render/Railway
- OBS Dessa tjänster erbjuder EN månad gratis bara eller x antal credits. (Tar dessa credits slut så kommer ni inte åt er tjänst, så ett tips är att deploya göra det som SISTA STEG.

Krav för Väl Godkänt (VG)

För att få betyget Väl Godkänt (VG) ska gruppen först uppfylla alla krav för G och dessutom alla krav nedan.

1. En tredje tjänst

- Ni ska bygga EN tredje microservice
- Håll den LITEN! Den ska göra EN sak (Se nedan)
- Egen applikation, egen databas, pratar med de andra tjänsterna via REST
- Välj EN av dessa (eller föreslå en egen idé för mig):
 - Recensionsservice – gäster kan lämna omdömen och betyg på rum de bokat
 - Notifieringsservice – skickar/loggar bokningsbekräftelser till kunder (Svårare)

2. JWT mellan tjänsterna

- Kundtjänsten ska ha en loginendpoint som ger användaren en JWTtoken
- Bokningstjänstens endpoints som ändrar data (skapa, ändra, avboka) ska kräva en giltig token
- Token skickas som en header i anropet: Authorization: Bearer <token> (enklaste sättet att göra det på, vill ni köra en annan variant går det bra också)
- När en tjänst anropar en annan tjänst ska token skickas med i anropet
- Håll det enkelt: Inga roller, ingen refresh token

3. Kubernetes

- Varje tjänst ska ha en Deployment och en Service (yamlfiler)
- Lägg alla yamlfiler i en mapp som heter k8s i ert repo
- Systemet ska gå att köra lokalt i Kubernetes (minikube eller Kubernetes i Docker Desktop)
- Hela systemet ska starta med: kubectl apply f k8s/ (Alltså alla 3 tjänster med deras databaser)

Tekniska krav (FÖR BÅDE G OCH VG)

Dessa krav gäller hela projektet.

1. Struktur i projektet

- ALLA tjänster ska ha controllerlager, servicelager och repositorylager (Multi tier)
- Koden ska ha en tydlig och logisk struktur

2. Servicelager

- Servicelagret ska innehålla all affärslogik, precis som i Backend 1

3. Validering och felhantering

- API:erna ska ge tydliga felmeddelanden med rätt statuskoder
- Validering ska användas för att stoppa ogiltig inmatning
- Exempel på sådant som ska hanteras tydligt:
 - ogiltiga inmatningar
 - försök att dubbelboka rum på samma datum
 - försök att boka åt en kund som inte finns
 - försök att ta bort en kund med aktiva bokningar
 - den andra tjänsten är nere

4. Tester

- Minst 3 integrationstester ska skrivas
- Integrationstesterna ska testa riktiga APIanrop.
- Exempel: anropa endpointen som skapar en bokning och kontrollera att svaret blir 201 och att bokningen hamnar i databasen.
- Era enhetstester från Backend 1 får vara kvar, men de räknas inte som integrationstester
- Det räcker inte att skriva tomma eller meningslösa tester bara för att nå antalet

Arbetsprocess och samarbete

Denna uppgift genomförs i samma grupper som i Backend 1.

Gruppen ska:

- planera arbetet tillsammans innan utvecklingen börjar
- dela upp arbetet i mindre delar så att alla vet vad de ansvarar för
- ha löpande avstämningar under projektets gång
- gärna ha korta dagliga möten eller regelbundna avstämningar

Viktigt

- Alla gruppmedlemmar SKA bidra aktivt
- En person ska INTE göra allt arbete

INLÄMNINGSUPPGIFT Individuellt (G)

Reflektion

Varje student ska svara på följande frågor med egna ord:

Om tekniken:

- Vad är en monolit och vad är en microservice? Vad är skillnaden?
- Varför delade ni upp pensionatet i flera tjänster? Vilka fördelar och nackdelar märkte ni själva?
- Varför har varje tjänst en egen databas? Varför får tjänsterna inte läsa i varandras databaser?
- Vad händer i ert system om kundtjänsten är nere? Hur hanterar ni det?
- **ENDAST VG:** skriv även kort om hur JWT fungerar i ert system.

Omfattning

- Rapporten behöver inte vara längre än 2 A4sidor totalt

Inlämning

- Lämna in länk till gruppens repository (eller repositories om ni har flera)
- Repositoriet ska vara publikt
- Det ska finnas en README som förklarar: (**Helt okej om ni använder AI för den här delen**)
 - vad varje tjänst gör
 - hur tjänsterna pratar med varandra
 - hur man startar hela systemet (docker compose up)
- Varje student lämnar dessutom in sin egen individuella rapport

Tips på byggordning

Steg 1 – Få igång Backend 1projektet

- Kontrollera att applikationen startar utan fel
- Rätta buggar som stör men lägg inte dagar på att göra gammal kod perfekt! (Finns ingenting som heter perfekt kod!)

Steg 2 – Skapa kundtjänsten

- Skapa ett helt nytt Spring Bootprojekt
- Använd en annan port än bokningstjänsten (till exempel 8081) de ska köras samtidigt

Steg 3 – Bygg kundtjänstens databas och API

- Skapa entity, repository, service och controller för kund

- Låt databasen skapas enligt code first precis som i Backend 1
- Bygg alla kundendpoints (hämta, registrera, ändra, ta bort)
- Testa alla endpoints i Postman/thunderclient innan ni går vidare

Steg 4 – Flytta kundhanteringen

- Kundsidorna i er frontend ska nu använda kundtjänsten istället för den gamla endpointen
- Kontrollera att det går att registrera, ändra och ta bort kunder via frontend

Steg 5 – Ta bort kundtabellen ur bokningstjänsten

- Ta bort kundentityn och kundtabellen ur bokningstjänsten
- Ändra bokningen så att den bara sparar kundens id (ett nummer)
- OBS detta steg är lite känsligt, det är meningen! Ni förvaltar ett system nu!

Steg 6 – Koppla ihop vid bokning

- När en bokning skapas ska bokningstjänsten fråga kundtjänsten: finns den här kunden?
- Finns kunden inte: ingen bokning, tydligt felmeddelande

Steg 7 – Koppla ihop vid borttagning av kund

- När en kund ska tas bort ska kundtjänsten fråga bokningstjänsten: har kunden aktiva bokningar?
- Har kunden aktiva bokningar: ingen borttagning, tydligt felmeddelande (precis som i Backend 1)

Steg 8 – Felhantering mellan tjänsterna

- Stäng av kundtjänsten och försök skapa en bokning – vad händer?
- Tjänsten ska inte krascha! den ska ge ett tydligt felmeddelande

Steg 9 – Skriv integrationstester

- Skriv minst 3 integrationstester
- Exempel på sådant ni kan testa:
 - att skapa en bokning ger 201
 - att dubbelbokning ger 409
 - att registrera en kund ger 201
 - att felaktig inmatning ger 400

Steg 10 – Docker

- Skriv en Dockerfile för bokningstjänsten och bygg en image
- Gör samma sak för kundtjänsten

- Kontrollera att båda går att starta som containrar

Steg 11 – Docker Compose

- Skapa en dockercompose.yml med båda tjänsterna och databaserna
- Testa docker compose up hela systemet ska starta

Steg 12 – VG: Bygg den tredje tjänsten

- Nytt projekt, egen databas, egen port
- Lägg till den i dockercompose.yml

Steg 13 – VG: Lägg till JWT

- Skapa en loginendpoint i kundtjänsten som returnerar en token
- Skydda bokningstjänstens endpoints som ändrar data
- Skicka med token när tjänsterna anropar varandra
- Testa i Postman: först utan token (ska stoppas), sedan med token (ska fungera)

Steg 14 – VG: Kubernetes

- Skriv Deployment och Serviceyaml för varje tjänst
- Lägg filerna i mappen k8s
- Starta minikube (eller Kubernetes i Docker Desktop)
- Kör kubectl apply f k8s/ och kontrollera att alla poddar startar
- Kontrollera att tjänsterna hittar varandra via sina Servicenamn